

AI-SA-IN-TH-000007

Module 8.1 · LLM Evaluation Fundamentals · AI Safety: Evaluation, Guardrails & Responsible Deployment

Output Guardrails

Toxicity Filtering, Factuality Checking, PII Redaction, and Confidence-Based Routing

9 min

Duration

B3 – Apply

Bloom's

L3 – Advanced

Level

Theory

Domain

Learning Objectives

- 1 Explain why output guardrails are necessary even when input guardrails are in place
- 2 Classify the five output harm types and describe the appropriate guardrail for each
- 3 Configure a toxicity classifier threshold using the precision-recall framework
- 4 Implement PII detection and redaction in generated text using NER-based techniques
- 5 Distinguish factuality guardrails from fluency guardrails and explain why they require different tools
- 6 Design a confidence-based routing system that escalates uncertain outputs to human review

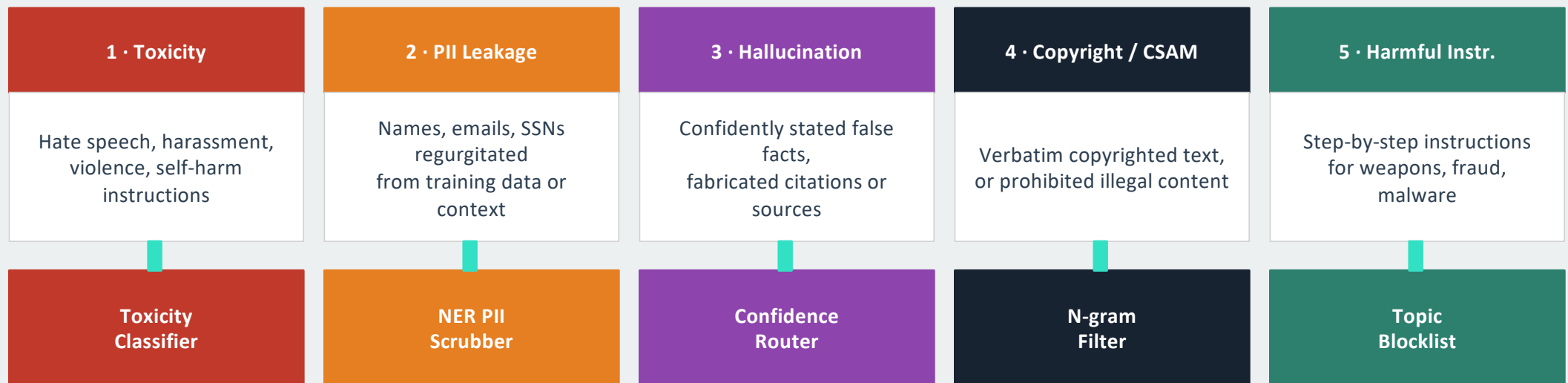
Why Output Guardrails Are Necessary

Output Guardrail

A software component that intercepts every model-generated response before delivery to the user, inspects it against safety, privacy, and quality criteria, and either passes it, modifies it, blocks it, or routes it for human review — operating independently of the model's own training.

- Input screening is necessary but not sufficient: a clean input can still produce a harmful output due to model failures
- Models can hallucinate PII from training data even when the user prompt contained no PII
- A model aligned not to produce toxicity can still generate it under creative or hypothetical framing that passed input checks
- Output guardrails close the gap between what the model was trained to do and what it actually produces in production
- Regulatory requirement: GDPR Art. 25 (data protection by design) and EU AI Act require output-level privacy controls

Five Output Harm Types — Taxonomy



Guardrail

Each harm type requires a purpose-built guardrail — no single classifier covers all five dimensions.

Toxicity Detection — Classifiers & APIs

Toxicity Classifier

A fine-tuned text classification model that assigns a probability score (0–1) to a piece of text indicating how likely it is to violate a toxicity policy (hate speech, harassment, violence, self-harm). Output guardrails use this score against a configurable threshold to decide pass / flag / block.

- Perspective API (Google Jigsaw): free REST API; scores: TOXICITY, SEVERE_TOXICITY, INSULT, THREAT, IDENTITY_ATTACK — 9 ms median latency
- Detoxify (HuggingFace): open-source fine-tuned BERT; runs locally — no API costs or data-sharing concerns
- OpenAI Moderation API: multi-label classifier returning scores across: hate, harassment, self-harm, sexual, violence, illegal
- Azure AI Content Safety: enterprise-grade; tiered severity scores (0–7); multi-modal (text + image); HIPAA compliant
- Selection criteria: Perspective for public-facing products; Detoxify for self-hosted/privacy-sensitive; Azure for regulated industries

Perspective API & Detoxify — Integration

Python

```
# Option A: Perspective API (cloud)
from googleapiclient import discovery
def perspective_toxicity(text: str, threshold=0.75) -> dict:
    client = discovery.build('commentanalyzer', 'v1alpha1',
                             developerKey=os.environ['PERSPECTIVE_KEY'])
    body = {'comment':{'text':text}, 'languages':['en'],
            'requestedAttributes':{'TOXICITY':{}, 'IDENTITY_ATTACK':{}} }
    resp = client.comments().analyze(body=body).execute()
    score = resp['attributeScores']['TOXICITY']['summaryScore']['value']
    return {'score':score, 'block': score > threshold}

# Option B: Detoxify (local, no data-sharing)
from detoxify import Detoxify
model = Detoxify('original', device='cpu') # load once
def detoxify_check(text: str, threshold=0.75) -> dict:
    results = model.predict(text)
    score = results['toxicity']
    return {'score': round(score,4), 'block': score > threshold}
```

Use Detoxify for self-hosted deployments where sending user data to Google violates data processing agreements. Load model once at startup — subsequent calls are < 30 ms on CPU.

Threshold Tuning — Precision vs Recall Framework

$$F_{\beta} = (1 + \beta^2) \cdot P \cdot R / (\beta^2 \cdot P + R) \quad \text{where } \beta \text{ controls P/R weight}$$

1. At threshold 0.5: Precision=0.72, Recall=0.94 — catches most toxicity but 28% of blocks are false positives (FP)
2. At threshold 0.75: Precision=0.89, Recall=0.81 — balanced; recommended for general consumer products
3. At threshold 0.90: Precision=0.97, Recall=0.62 — misses 38% of toxic content; only use when false positives are very costly
4. Use $\beta > 1$ (e.g. F_2) for safety-critical apps: weights recall higher → catches more toxicity, accepts more FP
5. Use $\beta < 1$ (e.g. $F_{0.5}$) for creative apps: weights precision higher → fewer false blocks of legitimate creative content

Note: A/B test threshold changes with 1% of traffic before rolling out. Even 1pp threshold change can shift block rate 10–30%.

PII Detection in Generated Outputs

Output PII Detection

The process of scanning model-generated text for personally identifiable information — including data regurgitated from training memory, from in-context documents, or inadvertently generated — and redacting or blocking it before delivery to the user or storage.

- Training data memorisation: LLMs trained on internet data have memorised real names, emails, and phone numbers from crawled content
- Context window leakage: if a user's PII appears in the system prompt, the model may echo it back to other users in multi-tenant deployments
- Generation artefacts: models sometimes generate plausible-but-fake PII that could still cause user confusion or privacy concerns
- Regulatory obligation: GDPR Art. 17 (right to erasure) requires that PII not be propagated in system outputs
- Detection method: same NER pipeline as input (Presidio/spaCy) — apply to full response text before delivery

NER-Based PII Redaction — Output Pipeline

Python

```
from presidio_analyzer import AnalyzerEngine
from presidio_anonymizer import AnonymizerEngine
from presidio_anonymizer.entities import OperatorConfig

analyzer = AnalyzerEngine()
anonymizer = AnonymizerEngine()

# Custom operators: replace with type label rather than <ENTITY>
operators = {
    'PERSON': OperatorConfig('replace', {'new_value': '[NAME]'}),
    'EMAIL_ADDRESS': OperatorConfig('replace', {'new_value': '[EMAIL]'}),
    'PHONE_NUMBER': OperatorConfig('replace', {'new_value': '[PHONE]'}),
    'US_SSN': OperatorConfig('replace', {'new_value': '[SSN]'}),
}

def scrub_output_pii(response: str) -> tuple[str, bool]:
    results = analyzer.analyze(text=response, language='en')
    if not results:
        return response, False
    clean = anonymizer.anonymize(response, results, operators).text
    return clean, True  # (scrubbed_text, was_modified)
```

Log `was_modified=True` events to your security dashboard. Spike in output PII detections signals a prompt injection or context leakage incident.

Factuality vs Fluency — Different Guardrail Dimensions

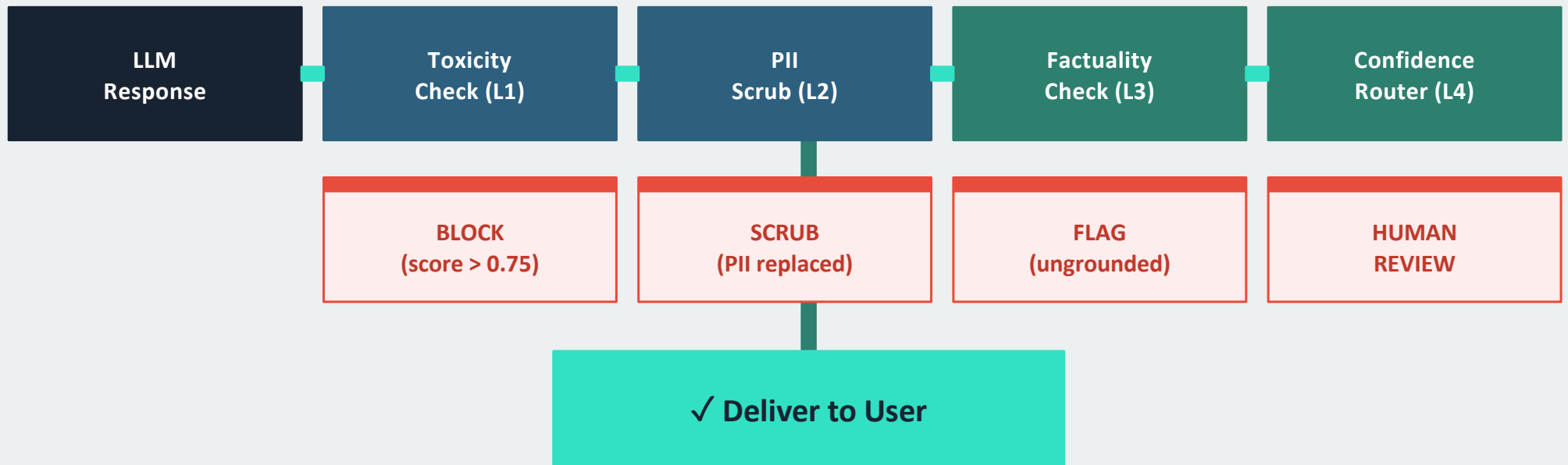
Fluency Guardrails

- Ensures output is grammatically correct, coherent, and appropriate in tone
- Detected by: perplexity scoring, coherence classifiers, length checks
- Low severity: fluency failures are annoying but rarely cause harm
- Typically auto-corrected via post-processing rather than blocked
- Example: truncated responses, off-language replies, incoherent paragraphs
- Tool: model's own log-probability scores; low avg log-prob → potential failure

Factuality Guardrails

- Ensures output does not contain verifiably false claims presented as facts
- Detected by: RAG grounding check, knowledge base lookup, citation verification
- High severity: factual errors in medical, legal, or financial contexts cause real harm
- Typically blocked or routed to human review on failure
- Example: model hallucinates a drug interaction, fabricates a legal citation
- Tool: compare claim against retrieved reference docs; flag claims without support

Multi-Layer Output Guardrail Pipeline



Short-circuit: if L1 blocks, skip L2–L4. Parallel L2+L3 reduces latency by 35%.

Output Guardrail Tool Comparison

Tool	Covers	Latency	Cost	Deployment	Best For
Detoxify (OSS)	Toxicity	< 30 ms	Free	Self-hosted	Privacy-sensitive, regulated
Perspective API	Toxicity + identity	~9 ms	\$0.001/req	Cloud	Consumer apps, social platforms
OpenAI Mod API	6 harm categories	~50 ms	Free	Cloud	OpenAI model deployments
Azure AI Content Safety	Text + image, tiered	~80 ms	~\$0.001	Cloud/BYOK	Enterprise, HIPAA, finance
Presidio (OSS)	PII (40+ types)	2–8 ms	Free	Self-hosted	GDPR/CCPA PII scrubbing
Guardrails AI (OSS)	Multi-type pipeline	50–150 ms	Free	Self-hosted	Custom policy enforcement

Case Study — Training Data Memorisation: Samsung & ChatGPT PII Leak

Case: Samsung Engineers Upload Confidential Source Code to ChatGPT (March 2023)

- Samsung Semiconductor engineers uploaded confidential source code and internal meeting notes to ChatGPT prompts to ask for debugging assistance and summarisation
- Three separate incidents within a month: chip yield data, meeting minutes about facility issues, and complete source code for a battery-measuring tool
- By pasting proprietary data into ChatGPT, Samsung engineers sent it to OpenAI servers — OpenAI's terms of service at the time allowed using data for model improvement
- The data could potentially surface in future model outputs if training included the conversation history
- Samsung banned internal use of generative AI tools after the incidents were discovered

Key Lesson

Output guardrails would not have prevented the data upload, but input guardrails with PII/IP detection on the input (Layer 2) would have blocked the source code and flagged the confidential data before it reached the model. This is an input + data governance failure, not an output failure — but it illustrates why both layers matter.

Confidence-Based Routing — Worked Example

High-uncertainty outputs are routed to human review rather than delivered directly

Medical Chatbot Response (without routing)

User: 'Can I take ibuprofen with warfarin?'

Model response: 'Yes, ibuprofen can generally be taken with warfarin at standard doses.' [Score: avg log-prob = -3.8 — low confidence]

DELIVERED DIRECTLY — but this is medically dangerous advice. Ibuprofen + warfarin significantly increases bleeding risk.

With Confidence-Based Routing

Step 1: Compute avg log-probability of response tokens = -3.8 (below threshold -3.0)

Step 2: Router detects low confidence → escalate to human reviewer

Step 3: Pharmacist reviews within 4 hours → BLOCK original response

Step 4: Deliver verified response: 'NSAIDs like ibuprofen significantly increase bleeding risk when combined with warfarin. Consult your prescribing physician before combining these medications.'

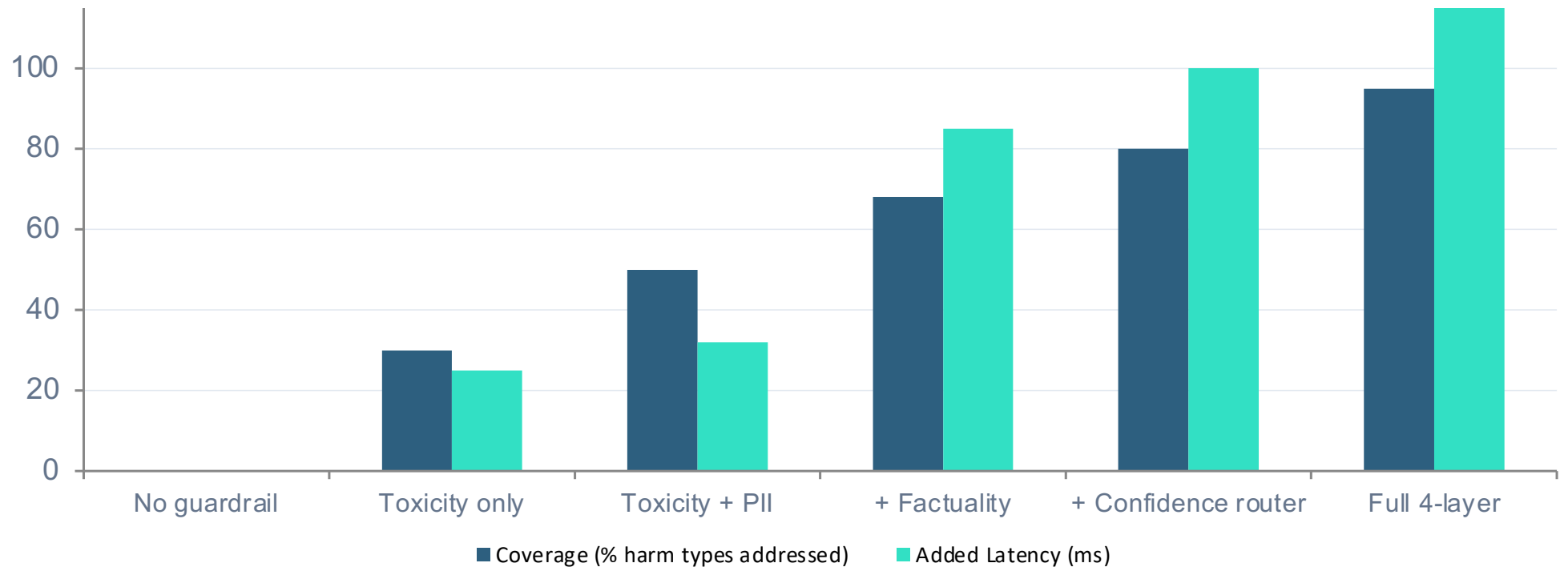
Conclusion: Confidence routing prevented a medically dangerous hallucination from reaching the user.

Output Watermarking & AI Provenance (C2PA)

Emerging standard: cryptographically sign AI-generated content to enable downstream verification:

- C2PA (Coalition for Content Provenance and Authenticity): open standard co-developed by Adobe, Microsoft, Google, Arm — embeds cryptographic provenance in content metadata
- AI-generated text watermarking: embed statistical patterns in token selection during decoding (Kirchenbauer et al., 2023) — detectable without access to the model
- Green-list watermarking: at each position, pseudo-randomly partition vocabulary into 'green' (permitted) and 'red' (restricted) tokens; watermarked text overuses green tokens
- Limitation: hard watermarks are fragile under paraphrase; soft watermarks (statistical) survive paraphrase but degrade generation quality
- Current status (2026): C2PA metadata is now attached to outputs from Adobe Firefly, OpenAI DALL·E, and Microsoft Copilot — text watermarking remains research-stage
- Use case: detecting AI-generated misinformation, verifying content provenance in news workflows, compliance with EU AI Act transparency requirements

Guardrail Latency vs Coverage Tradeoff



Async parallel execution of toxicity + PII keeps added latency below 40 ms. Factuality and confidence routing add ~85 ms — justify only for high-stakes domains (medical, legal, financial).

Key Takeaways

AI-SA-IN-TH-000007

- 1 Output guardrails are necessary because a clean input can still produce a harmful output — model alignment alone cannot guarantee safe outputs at production scale
- 2 The five output harm types (toxicity, PII leakage, hallucination, copyright, harmful instructions) each require a purpose-built guardrail — no single classifier covers all five
- 3 Threshold tuning for toxicity classifiers uses the precision-recall framework: F_2 for safety-critical apps (catch more attacks), $F_{0.5}$ for creative apps (fewer false blocks)
- 4 PII in outputs can originate from three sources: training data memorisation, context window leakage, and model generation artefacts — all three require NER-based output scrubbing
- 5 Factuality guardrails (RAG grounding check) and fluency guardrails (log-probability routing) are distinct tools for distinct harm types — do not conflate them